BACKEND TECHNICAL TASK

PROGINEER ENGINEERING CHALLENGE

Objective: Build a backend RESTful API using **.NET / ASP.NET Core** for a simple **Donation Management System**. The system allows users to create and manage donations through clean RESTful API endpoints while enforcing data privacy for anonymous donors.

1 DONATION MANAGEMENT & PRIVACY

A user should be able to add and manage donations in the system. Each donation record must store:

- **Donation Amount:** Numerical value representing the contribution.
- **Donor Name:** String field representing the donor's identity.
- **Anonymous Option:** Boolean flag indicating whether the donation is anonymous.
- **Creation Date/Time:** Timestamp recorded when the donation is created.

CRITICAL ANONYMITY REQUIREMENT: If a donation is anonymous, the donor's name MUST NOT be exposed in any API responses.

2 CRUD OPERATIONS & REST ENDPOINTS

Implement complete CRUD operations for donations using standard HTTP methods and status codes:

Method	Endpoint Route	Description	Expected HTTP Status
GET	`/api/donations`	Get all donations (masking anonymous names)	`200 OK`
GET	`/api/donations/{id}`	Get specific donation by ID	`200 OK` / `404 Not Found`
POST	`/api/donations`	Create a new donation	`201 Created` / `400 Bad Request`
PUT	`/api/donations/{id}`	Update an existing donation	`200 OK` / `204 No Content` / `404`
DELETE	`/api/donations/{id}`	Delete an existing donation	`200 OK` / `204 No Content` / `404`

3 HEALTH API

Create a dedicated health endpoint at `GET /api/health`. It should return a successful response when the API is running correctly, indicating that the system is healthy.

4 AUTHENTICATION & AUTHORIZATION SCOPE

IGNORE AUTHENTICATION COMPLETELY: Do not implement Login, Registration, JWT, Authentication, Authorization, Roles, or Permissions. All API endpoints must be publicly accessible without authentication.

5 SWAGGER / OPENAPI

The API must properly support Swagger/OpenAPI. Ensure that all endpoints are visible, request/response models are documented, status codes are represented, and endpoints can be tested directly in Swagger.

6 POSTMAN TESTING REQUIREMENTS

Create a Postman collection containing tests for all endpoints and validation scenarios:

Standard Test Scenarios

- **Health:** Verify health check response.
- **Create:** Normal & anonymous donations.
- **Read:** Get all & Get by ID (verify anonymous name masking).
- **Update:** Normal & anonymous donation updates.
- **Delete:** Delete record & verify subsequent access.

Validation & Error Scenarios

- Invalid donation amount (e.g., negative/zero).
- Missing required payload fields.
- Invalid or non-numeric donation ID.
- Requesting a non-existent donation ID.
- Updating or deleting a non-existent donation.

7 UI / FRONTEND BOUNDARY

> **Backend Only: Do not create any frontend or UI (no React, HTML, Razor pages, or UI components). Focus strictly on the .NET API implementation.**

8 TECHNICAL EXPECTATIONS

Use **ASP.NET Core / .NET**. Make clean implementation decisions regarding project structure, entity design, route naming, validation, HTTP status codes, error handling, separation of concerns, and database access. Keep the solution maintainable without over-engineering.

9 DATABASE ARCHITECTURE

Donation data must be persisted in a database (e.g., SQL Server, PostgreSQL, SQLite). You are responsible for deciding the database schema, table definitions, column types, primary keys, constraints, and indexes. Your design should be explainable.

10 DELIVERABLES CHECKLIST

Your final submission package must include:

1. Complete .NET backend project solution.
2. Database implementation (migrations/scripts).
3. Fully working CRUD APIs & Health endpoint.
4. Swagger/OpenAPI documentation integration.
5. Exported Postman collection containing test cases.
6. Comprehensive `README.md` detailing setup, database configuration, execution, Swagger access, Postman testing, and key design decisions.

ProGineer Technologies — WE DELIVER